



PROJECT DOCUMENTATION HANDOVER

COMPETE. IMPROVE. INSPIRE.

Version	2026.05 — Rebranded Edition
Platform	EVOLVE (formerly WARZONGG / WARZONE.GG)
Scope	Web · API · Supabase · Tournament Engines · Operations
Tech Stack	HTML · Tailwind · Vanilla JS · Node.js · Express · Supabase · PostgreSQL
Frontend	MVP COMPLETE — multi-page static application
Backend	IMPLEMENTED — architecture documented and operational
Author	Mlungisi Nxumalo + Claude (Anthropic)
Date	May 2026

*Complete reference for continuing development across sessions and with different AI assistants. Rebranded from WARZONE.GG / WARZONGG to **EVOLVE** — 2026 edition.*

Table of Contents

1. Brand, Positioning, and Product Purpose
2. User Roles and Access Model
3. Current Feature Inventory
4. System Architecture Overview
5. API Reference (Current)
6. Supabase Data Model
7. Tournament Engines and Scoring Behaviour
8. Frontend Runtime Configuration
9. Security and Access Controls
10. Admin Operations Playbook
11. Environment Variables (Backend)
12. Known Gaps and Technical Notes
13. Rebranding Checklist
14. Suggested Next Documentation Assets
15. Key Data Flows
16. Quick Reference Card

1. Brand, Positioning, and Product Purpose

1.1 Current Brand Identity

Field	Value
Primary brand name	EVOLVE
Legacy names in code / history	WARZONGG, WARZONE.GG
Tagline in product UI	Multi-Game Tournament Platform (South Africa)
Slogan	COMPETE. IMPROVE. INSPIRE.
Primary audience	Competitive gamers, team captains, admins, and tournament organisers

1.2 Updated Platform Purpose

EVOLVE is now a **multi-game competitive operations platform**, not only a single-title tournament site. The platform exists to:

- Onboard players and captains.
- Register squads into tournaments.
- Approve/reject entries through admin workflows.
- Run live and scheduled matches with score resolution logic.
- Present public-facing schedules, leaderboards, brackets, and announcements.

1.3 Product Direction & Supported Games

- COD Mobile
- COD Warzone
- COD Modern Warfare
- FIFA
- Need for Speed
- Forza
- Mortal Kombat
- Other (fallback)
- Strong split between public experience and admin operations.
- Tournament participation gating (pending / approved / rejected).
- COD map/mode resolution and best-of series handling.
- FIFA match outcome logic including extra-time and penalty pathways.

2. User Roles and Access Model

Role	Signed In?	Key Capabilities
Public user	No	Browse tournaments, schedule, leaderboard, announcements, bracket, and rules. Cannot register.
Player / Captain	Yes	Create account; register new team + roster; register existing captain-owned team to open tournament.
Admin	Yes (admin role)	Access admin dashboard and admin-only API routes; create/update/delete tournaments; create a

3. Current Feature Inventory

3.1 Public-Facing Pages

File	Description
index.html	Home — live banner, featured tournaments, activity feed, announcements
pages/tournaments.html	Tournament listings with filters and modal detail view
pages/tournament-hub.html	Individual tournament hub page
pages/bracket.html	Live bracket viewer with real-time score indicators
pages/leaderboard.html	Sortable team standings with KD, points, earnings, podium
pages/schedule.html	Day-grouped match schedule with status badges
pages/team-profile.html	Dynamic team page with roster, match history, stats
pages/player-profile.html	Individual player profile surface
pages/register.html	Multi-step team registration form
pages/activate.html	Player activation path for pre-registered roster emails
pages/my-team.html	Captain / player team management surface
pages/rules.html	7-section rulebook with anchor jump navigation

3.2 Admin-Facing Page

pages/admin.html — Single admin page with operational tabs:

- Dashboard overview
- Team registration triage (approve / reject)
- Match creation and result update
- Tournament management
- Announcements publishing
- Participant breakdown
- Live-site snapshot panel

3.3 Notable Workflow Enhancements

- Player activation path for pre-registered roster emails.
- Team captain notifications for team member departures.
- Existing-team tournament registration (without recreating team).
- Tournament participation tracking endpoint for signed-in users.

4. System Architecture Overview

Layer	Technology	Notes
Frontend	Static HTML/CSS/JS	Tailwind CDN + custom css/style.css. API integration via js/api.js. Runtime config via js/config.js
Backend API	Node.js + Express	Route prefix /api/*. JWT-based auth middleware. Admin authorisation middleware. Entitlements middleware
Data layer	Supabase Postgres + Auth	Two clients: supabase (anon key, RLS-respecting) and supabaseAdmin (service key, full access)
Frontend host	Netlify / Vercel / static	CORS controlled by env values and optional origin list.
Backend host	Render (or equivalent)	Free tier spins down after 15 min inactivity. Upgrade to Starter for live tournament days

5. API Reference (Current)

Base URL — Local: `http://localhost:3000` **Production:** `window.WZ_API_URL` (set in `js/config.js`)

5.1 Auth Routes

Method	Endpoint	Auth	Description
POST	<code>/api/auth/register</code>	None	Create new user account
POST	<code>/api/auth/login</code>	None	Returns JWT access token on success
POST	<code>/api/auth/activate-player/check</code>	None	Check pre-registered player email
POST	<code>/api/auth/activate-player</code>	None	Activate pre-registered player account
POST	<code>/api/auth/logout</code>	Auth	Invalidates current session token
GET	<code>/api/auth/me</code>	Auth	Returns current user profile and role

5.2 Tournament Routes

Method	Endpoint	Auth	Description
GET	<code>/api/tournaments</code>	None	List all tournaments (?status= and ?type= filters)
GET	<code>/api/tournaments/:id</code>	None	Single tournament detail with prize breakdown
GET	<code>/api/tournaments/:id/approved-teams</code>	None	Public list of approved teams for a tournament
POST	<code>/api/tournaments</code>	Admin	Create new tournament
PATCH	<code>/api/tournaments/:id</code>	Admin	Update tournament fields
DELETE	<code>/api/tournaments/:id</code>	Admin	Remove tournament and cascade-delete related data

5.3 Team / Registration Routes

Method	Endpoint	Auth	Description
POST	<code>/api/registrations</code>	Auth	New team or existing-team registration
GET	<code>/api/registrations</code>	Admin	List all registrations (?status= filter)
PATCH	<code>/api/registrations/:id/status</code>	Admin	Approve or reject a team registration
GET	<code>/api/teams/:id</code>	None	Team profile with roster and match history
GET	<code>/api/teams/mine</code>	Auth	Current user's team
GET	<code>/api/teams/mine/notifications</code>	Auth	Unread notifications for captain
PATCH	<code>/api/teams/mine/notifications/read</code>	Auth	Mark notifications as read
PATCH	<code>/api/teams/:id</code>	Captain	Update team info (name, tag, logo)

5.4 Player Routes

Method	Endpoint	Auth	Description
GET	/api/players/me	Auth	Current player profile
GET	/api/players/me/participations	Auth	All tournament participations for current player
POST	/api/players/me/leave-team	Auth	Leave current team (non-captains; roster integrity check)

5.5 Match and Series Routes

Method	Endpoint	Auth	Description
GET	/api/matches	None	List matches (?tournament_id= and ?round= filters)
GET	/api/matches/:id	None	Single match detail
POST	/api/matches	Admin	Submit a new match result
PATCH	/api/matches/:id	Admin	Correct a previously submitted result
POST	/api/match-series	Admin	Create a best-of series container

5.6 Leaderboard, Schedule & Announcements

Method	Endpoint	Auth	Description
GET	/api/leaderboard	None	Ranked teams by points (optional ?tournament_id=)
GET	/api/schedule	None	Upcoming and recent matches grouped by date
GET	/api/admin/dashboard	Admin	Overview stats: pending count, active tournaments
GET	/api/admin/teams	Admin	All teams with status filter and search
POST	/api/admin/announcements	Admin	Publish a new announcement
GET	/api/admin/announcements	Admin	List all announcements including unpublished
GET	/api/announcements	None	Public announcements feed

6. Supabase Data Model (Current Relevant Tables)

6.1 Table: users

Application profile row linked to Supabase Auth user. Auto-created on signup via database trigger.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Auto-generated by Supabase Auth
email	text	UNIQUE NOT NULL	Login email address
username	text	UNIQUE NOT NULL	Public display name
role	text	DEFAULT 'player'	Values: 'player' or 'admin'
whatsapp	text	NULLABLE	Contact number
discord	text	NULLABLE	Discord tag
player_id	uuid	NULLABLE	Link to players row if activated
created_at	timestampz	DEFAULT now()	Account creation time

6.2 Table: players

Roster/player identity rows linked to teams and optionally to user accounts.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique player row ID
team_id	uuid	REFERENCES teams(id)	Team this player belongs to
user_id	uuid	NULLABLE	Links to user account if registered
ign	text	NOT NULL	In-game name (IGN)
codm_uid	text	NOT NULL	3–32 chars [A-Za-z0-9._] — validated by SQL migration
email	text	NULLABLE	Contact email for activation
role	text	DEFAULT 'player'	igl / fragger / sniper / support / rusher / substitute
device	text	DEFAULT 'android'	android, ios, or emulator
is_substitute	boolean	DEFAULT false	Whether this is the sub slot

6.3 Table: teams

Team profile and performance aggregate row.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique team ID
name	text	UNIQUE NOT NULL	Full team name

Column	Type	Constraints	Description
tag	text	NOT NULL	Short tag (max 4 chars)
region	text	NOT NULL	Home province (e.g. Gauteng)
captain_id	uuid	REFERENCES users(id)	The user who registered the team
logo_url	text	NULLABLE	Supabase Storage URL for team logo
wins	integer	DEFAULT 0	Total wins across all tournaments
losses	integer	DEFAULT 0	Total losses
points	integer	DEFAULT 0	Total season points
earnings	integer	DEFAULT 0	Total prize money in ZAR cents
created_at	timestampz	DEFAULT now()	Team registration date

6.4 Table: tournaments ← MOST IMPORTANT TABLE

Central table of the application. Every tournament event is a row here. All other tables (registrations, matches) reference this table.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique tournament ID
name	text	NOT NULL	Full event name
tag	text	NOT NULL	Short identifier
mode	text	NOT NULL	Game mode label (e.g. Ranked 5v5, Squad BR)
type	text	NOT NULL	Internal type key: ranked, br, duos, solo
status	text	DEFAULT 'upcoming'	upcoming / open / live / completed
format	text	NOT NULL	Format description (e.g. Hardpoint + S&D)
region	text	DEFAULT 'ZA'	Target region
start_date	date	NOT NULL	Tournament start date
reg_deadline	date	NOT NULL	Registration closes date
max_slots	integer	NOT NULL	Maximum number of teams allowed
registered_count	integer	DEFAULT 0	Auto-updated by trigger
prize_total	integer	NOT NULL	Total prize pool in ZAR cents
entry_fee	integer	DEFAULT 0	Entry fee per team in ZAR cents
prize_1st	integer	DEFAULT 0	1st place prize in ZAR cents
prize_2nd	integer	DEFAULT 0	2nd place prize in ZAR cents
prize_3rd	integer	DEFAULT 0	3rd place prize in ZAR cents

Column	Type	Constraints	Description
prize_mvp	integer	DEFAULT 0	MVP award in ZAR cents
description	text	NULLABLE	Long-form tournament description
winner_team_id	uuid	NULLABLE	Populated when status = completed
created_by	uuid	REFERENCES users(id)	Admin who created this tournament
created_at	timestamptz	DEFAULT now()	Creation timestamp
updated_at	timestamptz	DEFAULT now()	Last modification timestamp
default_series_best_of	integer	NULLABLE	Engine: default series length (1/3/5/7)
cod_scoring_config	jsonb	NULLABLE	Engine: COD custom scoring behaviour
fifa_scoring_config	jsonb	NULLABLE	Engine: FIFA rule overrides per tournament

■ Prize values stored in cents (integers) to avoid floating-point errors. Divide by 100 to display. Example: `prize_total = 5000000` → R 50,000.

6.5 Table: registrations

Registration join between team and tournament, plus approval workflow state.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique registration ID
tournament_id	uuid	REFERENCES tournaments(id)	Which tournament
team_id	uuid	REFERENCES teams(id)	Which team is applying
status	text	DEFAULT 'pending'	pending / approved / rejected
game_mode	text	NOT NULL	Selected game mode for this registration
device_type	text	DEFAULT 'android'	Captain's primary device
notes	text	NULLABLE	Admin notes on the registration
submitted_at	timestamptz	DEFAULT now()	When registration was submitted
reviewed_at	timestamptz	NULLABLE	When admin approved or rejected
reviewed_by	uuid	NULLABLE	Admin who reviewed

■ *UNIQUE constraint (tournament_id, team_id) — a team can only register once per tournament.*

6.6 Table: matches

Match records — legacy single match and series map records. A Bo3 series generates up to 3 rows. Bracket built by querying grouped by round.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique match ID

Column	Type	Constraints	Description
tournament_id	uuid	REFERENCES tournaments(id)	Parent tournament
team_a_id	uuid	REFERENCES teams(id)	First competing team
team_b_id	uuid	NULLABLE	Second team (null = TBD)
winner_id	uuid	NULLABLE	Auto-set when scores submitted
round	text	NOT NULL	group_stage / quarter_final / semi_final / grand_final
map_name	text	NULLABLE	e.g. Standoff, Rust, Nuketown
game_mode	text	NULLABLE	e.g. Hardpoint, S&D, Domination
score_a	integer	NULLABLE	Team A score
score_b	integer	NULLABLE	Team B score
status	text	DEFAULT 'scheduled'	scheduled / live / completed
scheduled_at	timestamptz	NULLABLE	Planned match start time
played_at	timestamptz	NULLABLE	Actual match completion time
submitted_by	uuid	NULLABLE	Admin who submitted the result
stats	jsonb	NULLABLE	Mode-specific stat payload
cod_mode	text	NULLABLE	COD-specific game mode label
series_id	uuid	NULLABLE	Parent match_series.id for map rows
map_number	integer	NULLABLE	Map index within a series
counts_for_standings	boolean	DEFAULT true	False for series child maps
fifa_leg	integer	NULLABLE	FIFA leg number (1 or 2)
fifa_aggregate_pair_id	uuid	NULLABLE	Groups both legs of a FIFA tie

6.7 Table: match_series

Best-of series container for COD-style multi-map matchups.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique series ID
tournament_id	uuid	REFERENCES tournaments(id)	Parent tournament
team_a_id	uuid	REFERENCES teams(id)	First team
team_b_id	uuid	REFERENCES teams(id)	Second team
best_of	integer	1 / 3 / 5 / 7	Series format
round	text	NOT NULL	Same round values as matches.round

Column	Type	Constraints	Description
status	text	DEFAULT 'open'	open / complete
winner_team_id	uuid	NULLABLE	Set when series completes
created_at	timestampz	DEFAULT now()	Creation timestamp
updated_at	timestampz	DEFAULT now()	Last update timestamp

6.8 Table: announcements

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique announcement ID
title	text	NOT NULL	Announcement headline
body	text	NOT NULL	Full announcement text
type	text	DEFAULT 'info'	info / live / update / warning
tournament_id	uuid	NULLABLE	If targeted at a specific event
published_by	uuid	REFERENCES users(id)	Admin author
created_at	timestampz	DEFAULT now()	Publication time

6.9 Table: notifications

Internal user notifications — currently used for captain notices when team members leave.

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Unique notification ID
user_id	uuid	REFERENCES users(id)	Recipient user
type	text	NOT NULL	Notification category
message	text	NOT NULL	Notification body text
read	boolean	DEFAULT false	Whether the user has seen it
created_at	timestampz	DEFAULT now()	Creation timestamp

7. Tournament Engines and Scoring Behaviour

7.1 COD-Oriented Flow

- Match outcome resolved from scores / stats payload.
- Supports mode-specific stat payload in matches.stats.
- Team standings updates applied for completed non-series matches.
- Best-of series: match_series stores container state; child maps use series_id.
- counts_for_standings = false on map rows within a series.
- Series recompute updates winner and completion status.

7.2 FIFA-Oriented Flow

- Tournament mode / type text infers FIFA tournaments.
- Supports regulation outcomes, draw handling rules by phase/config.
- Extra-time and penalties resolved via stats payload.
- Group-stage leaderboard: points, draws, goals for/against, goal difference.

7.3 Config-Driven Options

Column	Type	Purpose
tournaments.cod_scoring_config	jsonb	COD custom scoring behaviour overrides per tournament
tournaments.fifa_scoring_config	jsonb	FIFA rule overrides per tournament

8. Frontend Runtime Configuration

File: `js/config.js`

Variable	Purpose
<code>window.WZ_API_URL</code>	API host — defaults to local/dev. Always override in production.
<code>window.WZ_SUPABASE_URL</code>	Supabase project URL
<code>window.WZ_SUPABASE_ANON_KEY</code>	Supabase anon/public key (safe for browser use)

- The browser must NEVER use the service role key.
- Supabase on the frontend is used mainly for bracket realtime page compatibility.
- The REST contract remains through the Express API for all other operations.
- Placeholder production URL defaults must always be overridden per environment.

9. Security and Access Controls

9.1 Auth & Authorization Model

- Login via Supabase Auth credentials.
- Protected routes require: Authorization: Bearer
- requireAuth — gates all authenticated user endpoints.
- requireAdmin — gates all admin-only management routes.

9.2 Security Checklist

Item	Status	How to Fix if Failing
.env is in .gitignore	Must pass	echo '.env' >> .gitignore before first commit
CORS origin is not wildcard *	Must pass	Set origin: process.env.FRONTEND_URL in Express CORS config
Admin routes use requireAdmin	Must pass	Add requireAdmin to every /api/admin route
Passwords not stored in plain text	Auto (Supabase)	Supabase Auth uses bcrypt internally
SQL injection not possible	Auto (Supabase SDK)	Never use raw SQL string concatenation
SUPABASE_SERVICE_KEY server-side only	Must pass	Never import service key in frontend JS files
Input validation on all POST routes	Must implement	Use express-validator on every route accepting body data
Rate limiting on auth routes	Recommended	npm install express-rate-limit; add to /api/auth routes
HTTPS enforced	Auto (Netlify/Render)	Do not use plain HTTP in production

10. Admin Operations Playbook (Current Practical Flow)

Step	Action	Detail
1	Create tournament	Use admin dashboard Tournament Management tab.
2	Wait for registrations	Monitor Registration Triage tab for incoming pending entries.
3	Approve / reject	Review team details, then approve or reject with optional notes.
4	Create matches	Set round, teams, and scheduled time for approved teams.
5	Move match statuses	scheduled → live → completed as each match is played.
6	Publish announcements	Use 'live' type for homepage urgency. Use 'info' for general notices.
7	Move tournament lifecycle	Update status: upcoming → open → live → completed.

11. Environment Variables (Backend)

Required in warzongg-api/ .env:

Variable	Required?	Source / Notes
SUPABASE_URL	Required	Supabase > Project Settings > API
SUPABASE_ANON_KEY	Required	Supabase > Project Settings > API (public key)
SUPABASE_SERVICE_KEY	Required	Supabase > Project Settings > API (secret) — server-side ONLY
PORT	Optional	Defaults to 3000; Render sets automatically
FRONTEND_URL	Recommended	Your Netlify URL (e.g. https://evovle.netlify.app)
FRONTEND_URL_PROD	Recommended	Production domain if different from FRONTEND_URL
CORS_ORIGINS	Optional	CSV of additional allowed origins
RESEND_API_KEY	Optional	Transactional email notifications via Resend.com

■ **Never commit .env to GitHub. Add it to .gitignore immediately on project creation.**

12. Known Gaps and Technical Notes

Item	Detail	Priority
Missing engine modules	matchesController imports engine/cod and engine/fifa modules not present in current repo tree. Validate before merge.	High
Legacy naming in code	WARZONE.GG / WARZONGG naming remains in some files and env commands while UI brand is EVOLVE.	Medium
Placeholder production URLs	Frontend config still contains placeholder production URL defaults — must be overridden per environment.	High
codm_uid constraint	Validated by fix_players_codm_uid_check.sql (3–32 chars, [A-Za-z0-9._-]). Ensure migration applied in all envs.	Medium

13. Rebranding Checklist (Completed in This Document)

Status	Item
✓	Platform identity updated to EVOLVE as primary brand.
✓	Purpose rewritten to reflect current multi-game operations.
✓	Feature inventory aligned to actual pages and role flows.
✓	API section fully refreshed from live route/controller implementation.
✓	Supabase table documentation updated with current usage and SQL extensions.
✓	Match engine behaviour documented for COD and FIFA tracks.
✓	Operational/admin procedures modernised for current platform workflows.

14. Suggested Next Documentation Assets

Priority	Asset	Purpose
1	docs/api/openapi.yaml	Generated source-of-truth API contract — enables client SDK generation and automa
2	docs/db/schema-reference.md	Column-by-column canonical schema with constraints, indexes, and trigger document
3	docs/ops/runbook.md	Incident + release + tournament-day checklist for operators.
4	Quarterly docs refresh	Link docs refresh to release milestones to keep documentation current.

15. Key Data Flows

15.1 Team Registration Flow

Step	Actor	Action	Result
1	Captain (browser)	Fills out registration form and submits	POST /api/registrations sent to Express
2	Express	requireAuth middleware checks JWT	Request rejected if not logged in
3	Express	Validates all fields with express-validator	Returns 400 error with field details if invalid
4	Express	Checks tournament has open slots	Returns 409 error if tournament is full
5	Supabase	Creates teams + players rows in transaction	New team in DB with pending registration
6	Supabase	Creates registrations row — status = pending	Admin can now see it in dashboard
7	Express	Returns { success: true, registration_id }	Frontend shows success message
8	Admin	Reviews in Admin > Teams tab; approves	PATCH /api/registrations/:id/status
9	Supabase	Updates status; increments registered_count	Team officially in the tournament
10	Email	Confirmation email sent to captain	Captain receives entry fee payment instructions

15.2 Match Result Submission Flow

Step	Actor	Action	Result
1	Admin	Submits scores in Admin > Matches tab	POST /api/matches with scores, map, mode, round
2	Express	requireAdmin middleware — blocks non-admins	Only verified admin can submit results
3	Express	Determines winner_id from score_a vs score_b	winner_id set automatically
4	Supabase	Creates matches row — status = completed	Result stored permanently
5	Supabase Realtime	Broadcasts UPDATE to all subscribed clients	All browsers watching bracket update instantly
6	Frontend	channel listener calls updateBracketDisplay()	Bracket re-renders, no page refresh
7	Express trigger	Updates team wins / losses / points	Leaderboard reflects new standings

16. Quick Reference Card

16.1 Key URLs

Resource	URL
Frontend (Netlify)	_____.netlify.app
API (Render)	_____.onrender.com
Supabase Dashboard	https://supabase.com/dashboard/project/_____
GitHub — Frontend	https://github.com/_____/warzongg-frontend
GitHub — API	https://github.com/_____/warzongg-api
Custom Domain	warzone.co.za or _____

16.2 NPM Packages

Package	Version	Purpose
express	^4.18.0	Web framework — handles HTTP routes
cors	^2.8.5	Allows frontend to call the API
dotenv	^16.0.0	Loads .env variables into process.env
@supabase/supabase-js	^2.x.x	Official Supabase client library
express-validator	^7.x.x	Input validation middleware
express-rate-limit	^7.x.x	Rate limiting for auth routes
nodemon	^3.x.x	(dev only) Auto-restarts server on file changes

16.3 Admin Dashboard Credentials (MVP Demo Only)

Field	Value
Username	admin
Password	admin123
■ Note	MVP demo credentials only. Replace with Supabase Auth in production.

When continuing this project with a new AI assistant or in a new session, share this document. It contains everything needed to understand the platform from scratch: purpose, file structure, all database tables, API endpoints, security approach, and deployment instructions.